

Data cleaning and preparation

Week Five: Non-numeric data

Before class begins...

- Read Chapter 6 in your text.

Some leftovers from Chapter 5

There are three concepts I want to discuss from chapter 5 that are basic data cleaning problems.

One is **missing** or **empty** values. Those of you who have had the data mining course have run across this problem already, and we've seen them in the diabetes dataset.

We will talk about what to do with these values later in the course, but right now I wanted to make sure you are aware of them, and what the difference is between them.

Missing values are those we expect to see but don't. For example, we expect to see a zip code for a US address. If one isn't in the dataset, that's a missing value.

Concept 1: Missing vs. empty

Empty values occur in attributes that have no meaning for that particular instance. An example is the number of pregnancies; for a male, that's an empty value (or should be!).

Often, database designers (or dataset preparers) insert a placeholder value in empty cells. These values need to be chosen with care, and documented thoroughly. We can see why in the diabetes dataset.

Things get sticky when the database designer uses a placeholder that could be a valid entry, and they get even stickier if a placeholder is a possible value for one attribute but not for another, and yet is used throughout the dataset.

Missing vs. empty

In the diabetes dataset, a zero value for number of pregnancies is absolutely a valid number. However, a zero value for diastolic blood pressure and for skin fold thickness is not.

If you don't have relevant domain knowledge, you won't catch these problems. As a matter of fact, if you look at the description of the diabetes data it specifically says there are no missing values. If you didn't have some domain knowledge, you wouldn't know.

HOWEVER—not having domain knowledge doesn't absolve you if you use this dataset for research without checking the quality of the data.

Why are we missing data?

We can categorize missing data into three cases:

1. Missing Completely At Random (MCAR); in this case, the probability that a data value is missing is unrelated to the value of the attribute or the attribute itself. One piece of data has the same probability of being missing as any other piece.
2. Missing At Random (MAR): This case is a little harder to explain. Here, we look at the probability of an attribute value being missing, given the value of a *different* attribute. So if, in the auto database, we looked at only European cars, the probability of the horsepower value being missing should be no different than the probability of other attribute values (given we are looking at European cars) being missing.

Why are we missing data?

3. Missing Not At Random (MNAR); in this case, data are missing for several reasons—sensor dropout, refusal to answer a particular question, etc. Statistically speaking, this situation means we do not have unbiased data.

Looking at the pattern of missing data can help us identify why the data values are missing. With randomly missing data, we don't have as much of a problem of bias as we do with MNAR data.

We will talk later about what to do with missing values.

Concept 2: Constants and unique values

I mentioned before about attributes that have the same value throughout. The foundation of data mining is pattern-finding—so if nothing ever changes, there's really no pattern. These attributes are useless and you can discard them without a second thought.

The flip side of constant attributes are those with all unique values, or a large number of unique values. These attributes change all the time, and that causes problems because there are too many patterns. These attributes are candidates for cardinality reduction.

Concept #3: Monotonics

Monotonic attributes are a special case of uniquely-valued variables. These attributes are often record IDs, or time values. They exhibit one or two basic characteristics:

1. They increase (or decrease) without bound. There's no range limits, and the values may each be unique.
2. They appear in the sample at an almost constant rate: when you sample an instance, you often get a new value you haven't seen before.
3. They exhibit **interstitial linearity**.

Wait! That's three characteristics. (Cue the [Spanish Inquisition!](#))

More on monotonics

Interstitial linearity means there's approximately the same distance between successive values of the attribute. In the case of a record or transaction ID, that distance is 1 unit.

Can we throw monotonics out? Well, yes, if the attribute is something un-useful like a record/transaction ID. But if the attribute contains useful information—maybe we don't want to get rid of that. One thing we can do is make a new attribute that contains the interval between each value. For example, if a machine broke down at 1:30 and the next breakdown was at 1:34, the interval is 4 (Of course, if the interval is always four, that's not helpful).

On to chapter 6: remapping non-numerical variables

Those of you who have taken the data mining class know that algorithms usually cannot handle all types of data. If you've ever tried to run a Weka neural network on this [cylinder band](#) data, or on this [auto mpg](#) data, and included the customer name field or the auto make field, you would quickly realize that your neural net was never going to converge.

The reason for the difficulty was that Weka generated a node for *every unique value* of those fields, resulting in a huge neural net that couldn't effectively converge to a solution.

The difficulties with non-numerics

As we saw earlier in the course, non-numeric data types come in three “flavors”:

- Nominal data (just names of things)
- Categorical data (groups of things)
- Ordinal data (things that have some inherent order)

All three non-numeric types pose challenging problems for mining. How do we represent these variables in a numeric system? How do we keep the inherent order of ordinal data?

1 to N mapping

What Weka did with the nominal data was to create N binary attributes, where N is the number of values present in the categorical variable.

Suppose we had a variable called *ResidenceType* that takes the values Rent (R), Own (O), Live with family (WF), Shared accommodations (SA) and Unknown (U). We could generate five binary variables to capture this information:

	R	O	WF	SA	U
Rent	1	0	0	0	0
Own	0	1	0	0	0
Live with family	0	0	1	0	0
Shared accommodation	0	0	0	1	0
Unknown	0	0	0	0	1

1 to (N-1) mapping

But, if we think about it a minute, we really don't need five variables. These binary variables are linearly independent—so we could infer the fifth value from the other four.

	R	O	WF	SA
Rent	1	0	0	0
Own	0	1	0	0
Live with family	0	0	1	0
Shared accommodation	0	0	0	1
Unknown	0	0	0	0

Which value should we drop? Any one we want, but it does make sense to drop the value with the lowest frequency.

Cardinality reduction

When an attribute has too many values (a rule of thumb is more than 10-12 distinct values), it's a candidate for cardinality reduction techniques.

Refaat* mentions three basic questions we need to ask before embarking on a reduction effort:

1. What is the maximum acceptable number of values?
2. How do we reduce the number of values?
3. What are the potential issues when we deploy the mining solution?

Using state space

Pyle suggests using a numeric attribute to “calibrate” alpha variables and transform them into numeric values (see his example of the hockey player position, calibrated by player height and weight). This approach assumes there is a consistent relationship between all three variables.

What other examples (besides height, weight, and position) can you think of where this approach might work well?

Joint distribution tables

If we have several alpha variables, we can construct joint distribution tables to see the underlying relationships among them. These tables can help us figure out how to apply numeric values to the variables.

But does this approach always work? Let's look at the famous weather data set.

The weather data

Outlook	Temp	Humidity	Wind	Play
sunny	hot	high	FALSE	no
sunny	hot	high	TRUE	no
overcast	hot	high	FALSE	yes
rainy	mild	high	FALSE	yes
rainy	cool	normal	FALSE	yes
rainy	cool	normal	TRUE	no
overcast	cool	normal	TRUE	yes
sunny	mild	high	FALSE	no
sunny	cool	normal	FALSE	yes
rainy	mild	normal	FALSE	yes
sunny	mild	normal	TRUE	yes
overcast	mild	high	TRUE	yes
overcast	hot	normal	FALSE	yes
rainy	mild	high	TRUE	no

The joint distributions are pairwise frequency tables. What we want to do is find overlaps that we can use to place the attributes in some sort of order *based on their relationship*.

Can we always use these tables? Obviously not. Sometimes the relationships are much more complex than we can represent in this way.

The weather data: joint distribution tables

Even though the weather dataset is pretty small, we have some complex relationships among the variables. In the book example, the author has set up the data so the overlaps—places where one variable does not have relationships with one or more other variables—fit nicely into a pattern.

	high	normal	Total
sunny	3	2	5
overcast	2	2	4
rainy	2	3	5
Total	7	7	14

	Windy	Not windy	Total
sunny	2	3	5
overcast	2	2	4
rainy	2	3	5
Total	6	8	14

	High	Normal	Total
Hot	3	1	4
Mild	4	2	6
Cool	0	4	4
Total	7	7	14

	Windy	Not windy	Total
Hot	1	3	4
Mild	3	3	6
Cool	2	2	4
Total	6	8	14

Going a bit deeper

When we look at the pairwise relationships in the weather data, we can see right away that all the instances where the temperature is cool occur when humidity is normal. However, there's one instance of normal humidity when the weather is hot. That makes it impossible to come up with a unique numeric ranking for these variables.

Sometimes that issue can be resolved by considering multiple relationships (combining humidity and wind and comparing with temperature, for instance).

Try determining a numeric value for humidity using the weather dataset that has a numeric temperature variable (see next slide).

Normal	High	High	High	High	High	High	High	Normal	Normal	Normal	Normal	Normal	Normal
Hot	Hot	Hot	Hot	Mild	Mild	Mild	Mild	Mild	Mild	Cool	Cool	Cool	Cool

Weather data with numeric attributes

Outlook	Temp	Humidity	Wind	Play
sunny	85	85	FALSE	no
sunny	80	90	TRUE	no
overcast	83	86	FALSE	yes
rainy	70	96	FALSE	yes
rainy	68	80	FALSE	yes
rainy	65	70	TRUE	no
overcast	64	65	TRUE	yes
sunny	72	95	FALSE	no
sunny	69	70	FALSE	yes
rainy	75	80	FALSE	yes
sunny	75	70	TRUE	yes
overcast	72	90	TRUE	yes
overcast	81	75	FALSE	yes
rainy	71	91	TRUE	no

A few other ideas

Outlook	Temp	Humidity	Wind	Play
sunny	hot	high	FALSE	no
sunny	hot	high	TRUE	no
overcast	hot	high	FALSE	yes
rainy	mild	high	FALSE	yes
rainy	cool	normal	FALSE	yes
rainy	cool	normal	TRUE	no
overcast	cool	normal	TRUE	yes
sunny	mild	high	FALSE	no
sunny	cool	normal	FALSE	yes
rainy	mild	normal	FALSE	yes
sunny	mild	normal	TRUE	yes
overcast	mild	high	TRUE	yes
overcast	hot	normal	FALSE	yes
rainy	mild	high	TRUE	no

Another approach is to use a frequentist method based on the relationship of the attribute with the target value. Let's consider the simple two-class case of the weather data. We want to transform the categorical variable *humidity* with respect to the target value *play*.

Frequentist method

Outlook	Temp	Humidity	Wind	Play
sunny	hot	high	FALSE	no
sunny	hot	high	TRUE	no
overcast	hot	high	FALSE	yes
rainy	mild	high	FALSE	yes
rainy	cool	normal	FALSE	yes
rainy	cool	normal	TRUE	no
overcast	cool	normal	TRUE	yes
sunny	mild	high	FALSE	no
sunny	cool	normal	FALSE	yes
rainy	mild	normal	FALSE	yes
sunny	mild	normal	TRUE	yes
overcast	mild	high	TRUE	yes
overcast	hot	normal	FALSE	yes
rainy	mild	high	TRUE	no

The relative frequency of *humidity* = high given *play* = yes is $3/7$ (number of times humidity is high and play is yes, divided by total humidity = high).

The relative frequency of *humidity* = normal given *play* = yes is $6/7$.

Handling ordinals

But humidity is really an ordinal value. High is greater than normal, although we don't know how much greater. So how can we preserve the ordering of the humidity variable?

A simple way to transform ordinals is to assume equal differences between the values and assign equally spaced numbers between 0 and 1 (or -1 and +1 for neural nets). For example, an ordinal with 14 distinct values would be assigned values of $\{-1, -0.86, \dots, +0.86, +1\}$.

Using concept hierarchies

If an alpha variable cannot be enumerated as is, another approach might be to try using higher abstraction, or moving up in the concept hierarchy. This approach removes some detail from the dataset, so use with caution.

What's a concept hierarchy? Suppose we have a transaction database such as those used in association mining.

Concept hierarchies

Association mining is also called “market basket analysis”; the database is a list of transactions and the items in the list are in no particular order.

TID	Produce
1	MILK, BREAD, EGGS
2	BREAD, SUGAR
3	BREAD, CEREAL
4	MILK, BREAD, SUGAR
5	MILK, CEREAL
6	BREAD, CEREAL
7	MILK, CEREAL
8	MILK, BREAD, CEREAL, EGGS
9	MILK, BREAD, CEREAL

This transaction database is at a high level of abstraction—we can break down “milk”, for example, into whole or 2% or 1% or skim or even soy milk. Even further, into Tops milk or Wegmans milk.

Concept hierarchies

We can do the same with cereal, bread, eggs—everything in this dataset. While we lose some detail going higher in the hierarchy, we can (possibly) simplify the relationships enough to be able to transform alphas into numbers.

Moving to a higher abstraction is, essentially, dimensionality reduction.

Dimensionality reduction

Another way to reduce dimensions is to combine attributes. When we work with numeric variables, we use principal components analysis to construct new features that capture the variability in the data. With non-numeric data, we combine attributes.

This approach works best when you have one attribute with values that always map to another—for instance, in the contact lens dataset the “reduced” value for tear production always maps to “None” for lens recommendation.

Contact lens dataset

Age	Prescription	Astigmatism	Tear production	Lens type
young	myope	yes	reduced	none
presbyopic	myope	yes	reduced	none
pre-presbyopic	myope	yes	reduced	none
young	hypermetrope	yes	reduced	none
presbyopic	hypermetrope	yes	reduced	none
pre-presbyopic	hypermetrope	yes	reduced	none
young	myope	no	reduced	none
presbyopic	myope	no	reduced	none
pre-presbyopic	myope	no	reduced	none
young	hypermetrope	no	reduced	none
presbyopic	hypermetrope	no	reduced	none
pre-presbyopic	hypermetrope	no	reduced	none
young	myope	yes	normal	hard
young	hypermetrope	yes	normal	hard
presbyopic	myope	yes	normal	hard
presbyopic	hypermetrope	yes	normal	none
pre-presbyopic	myope	yes	normal	hard
pre-presbyopic	hypermetrope	yes	normal	none
young	myope	no	normal	soft
young	hypermetrope	no	normal	soft
presbyopic	myope	no	normal	none
presbyopic	hypermetrope	no	normal	soft
pre-presbyopic	myope	no	normal	soft
pre-presbyopic	hypermetrope	no	normal	soft

Some other things to think about

If a level of a categorical variable appears rarely in the dataset, it acts more as noise to the algorithm. You can group this value together with others to reduce the dimensionality.

What if you have a value that occurs too often? In this scenario, there's little variability and that reduces the chance of finding a meaningful pattern. Going further down the concept hierarchy to a level where there is more variety to the number of values may help here.

Too many levels

If a categorical variable has too many levels, it approaches a rate of discovery that is analogous to a monotonic variable. Zip codes are a great example of a categorical variable with too many levels.

How can we handle this situation? With zip codes, we can map the values to locations and socioeconomic factors. That might increase the number of attributes we have, so be careful.

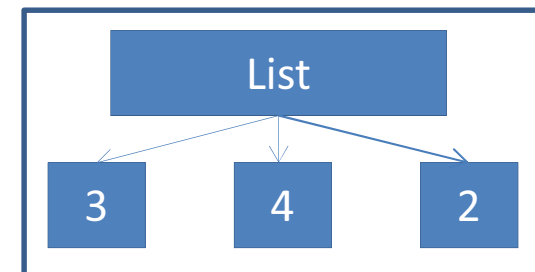
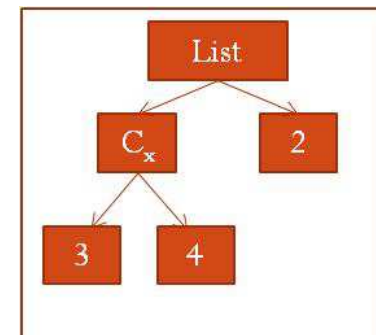
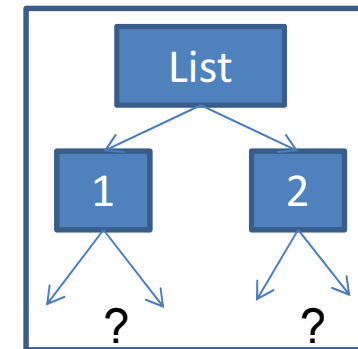
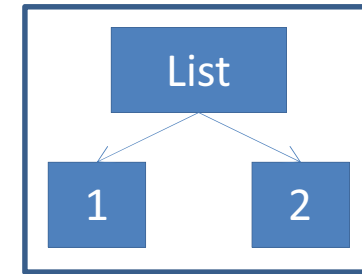
Structured grouping methods

The cardinality reduction method of choice is to form groups that make sense for the problem we are studying. But if we can't do that for some reason, we can use algorithmic methods to accomplish our task.

Decision tree algorithms split datasets based on some information function, such as entropy, chi squared value, or the Gini measure. Let's take a look at an algorithm that uses the Gini measure to split the dataset into subsets.

The basic algorithm*

1. Start with an empty list of terminal nodes.
2. Find a split, generating right and left nodes R and L .
3. Store these nodes in the terminal node list. The list now holds n terminal nodes.
4. If n is equal to the maximum required categories, stop.
5. Else, loop over all n nodes in the terminal node list.
6. Attempt to split each node and measure the information gained from that split.
7. Find the best split among the n candidate splits. Call this node C_x .
8. If the information gain is less than the minimum required gain, then stop.
9. Else, split node C_x into right and left nodes R and L . Delete C_x from the terminal node list and add R and L .
10. Set $n = n+1$.
11. Go to Step 4.



The Gini measure

The Gini measure represents the diversity, or inequality, of information in the data. It is commonly used in decision tree algorithms, CART in particular.

First let's set up some notation, and assume a binary dependent variable.

y = dependent variable (DV)

N = total number of records in the dataset

X = independent variable

N_1, N_0 = number of records for the categories 0 and 1 of y

N_1^j, N_0^j = number of records in the j^{th} child node with $y = 1$ and $y = 0$, respectively.

N_j = total number of records in the j^{th} child node

m = number of splits of the variable X

The Gini measure

The Gini measure of the parent node is $G_p = 1 - \frac{\sum_{k=0}^1 N_k^2}{N^2}$

The Gini measure for the j^{th} child node is $G_j = 1 - \frac{\sum_{k=0}^1 (N_k^j)^2}{(n_j)^2}$

Then, the weighted Gini measure for the current split on X is

$$\hat{G} = \sum_{j=1}^m \frac{n_j}{N} G_j$$

And finally, the Gini ratio is

$$G_r = 1 - \frac{\hat{G}}{G_p}$$

This ratio represents the percent increase in the information function we use in the algorithm.

Modifications to the measure

We assumed a binary dependent variable in this algorithm, but we can extend it to DVs with more than two values. For example, we replace the summation index for the parent node. Instead of ending at 1, the summation continues for c categories.

$$G_t = 1 - \frac{\sum_{k=0}^c N_k^2}{N^2}$$

Similarly, the Gini measure for the j^{th} node also sums over c categories.

The other modification is the split point; we need to search over all categories, not just 0 and 1.

The end result

The algorithm ends when we have the maximum number of groups, or the information function does not increase by the threshold amount. Although this procedure doesn't find the best grouping (like all partitioning algorithms, we end up working with smaller and smaller subsets), the ones it finds are better than the original, complete set of values.

Summary

Non-numeric variables pose some interesting challenges to the data miner and the mining algorithm. Sometimes we can exploit obvious relationships among non-numerics and numerics to map the categorical values to numbers. Sometimes we can't.

There's no substitute for a domain expert to help with these decisions. And remember, it doesn't hurt to try different approaches; one approach won't fit all datasets or domains.

Discussion for this week

Discussion question:

- Consider the [cylinder bands](#) dataset. Discuss two challenges posed by this dataset and what approaches might be useful in dealing with them.